

Beatriz São Leandro Cosimatti
17024130

João Victor Bozola Bosi
5979703

João Vitor Vasconcelos Jacob
16987258

Laís Antunes Cayres Quintão
16880392

Trabalho de Estrutura de Dados II:
Análise e Implementação de Algoritmos

Disciplina: Estrutura de Dados II
SCC0606
Professor: Renato M. Silva

São Carlos
14 de Junho de 2026

Sumário

1	Introdução	3
2	Algoritmos Base	3
2.1	Bubble Sort	3
2.2	Insertion Sort	3
2.3	Quick Sort	4
2.4	Heap Sort	5
2.5	Radix Sort	6
3	Arquitetura do Sistema	6
3.1	Execução da Aplicação	6
3.1.1	Seleção do Algoritmo	7
3.1.2	Geração Dinâmica de Vetores	7
3.1.3	Controle do Grau de Desordem	7
3.1.4	Leitura de Dados a Partir de Arquivos	8
3.1.5	Inserção Manual de Dados	8
3.1.6	Saída do Programa	8
3.2	Distribuições dos Vetores Utilizados	8
3.2.1	Vetor Aleatório	8
3.2.2	Vetor Quase Ordenado	8
3.2.3	Vetor Discrepante	9
3.2.4	Vetor Crescente	9
3.2.5	Vetor Decrescente	9
3.2.6	Distribuição Organ Pipe	9
3.2.7	Distribuição Intercalada	9
3.3	Dados de Desempenho Coletados	10
3.4	Métricas Coletadas	10
3.4.1	Grau de Desordem	10
3.4.2	Quantidade de Elementos Repetidos	10
3.4.3	Amplitude dos Valores (Range)	11
3.4.4	Desvio Padrão	11
3.4.5	Valores Máximo e Mínimo	11
3.5	Seleção das Métricas	11
3.6	Heurística Final	11
4	Validação das Heurísticas	12
4.1	Hipótese 1: Vetores Pequenos Favorecem o Insertion Sort	12
4.2	Hipótese 2: Vetores Quase Ordenados Favorecem o Insertion Sort	12
4.3	Hipótese 3: O Radix Sort Necessita de um Tamanho Mínimo	13
4.4	Hipótese 4: Vetores Grandes Favorecem o Radix Sort	13
4.5	Hipótese 5: Vetores Discrepantes Devem Evitar o Radix Sort	13
4.6	Hipótese 6: Vetores Muito Grandes Favorecem o Heap Sort	13
5	Metodologia Experimental	13
5.1	Ambiente de Execução e Ferramentas	13
5.2	Conjuntos de Dados (Datasets) e Distribuições	13
5.3	Volumetria e Tamanhos Testados	14
6	Resultados	14
6.1	Bubble Sort	14
6.2	Insertion Sort	16
6.3	Quick Sort	16
6.4	Heap Sort	19
6.5	Radix Sort	20
6.6	Validação da Heurística Final (Oráculo)	21

7	Reflexão sobre Uso de IA	22
7.1	Como a IA Foi Utilizada	22
7.2	Contribuições Positivas	23
7.3	Limitações e Erros Encontrados	23
7.4	Aprendizados Obtidos	23
8	Conclusão	23
9	Referências Bibliográficas	23

1 Introdução

O objetivo do trabalho desenvolvido foi estudar e implementar diferentes algoritmos de ordenação na linguagem C. Como cada algoritmo se comporta de uma maneira específica dependendo do tamanho do vetor ou de como os números estão organizados, o objetivo final do projeto é a criação de um Sistema Adaptativo. Esse sistema atua como um analisador que, antes de iniciar a ordenação, extrai métricas do vetor de entrada. A partir dessas informações e por meio de uma heurística de decisão, o sistema seleciona e executa de forma automatizada o algoritmo com maior potencial de eficiência para o cenário em questão, minimizando o tempo de execução e o uso de recursos computacionais.

Para fundamentar as tomadas de decisão desse sistema, o projeto foi estruturado de maneira modular, englobando a implementação de cinco algoritmos: Bubble Sort, Insertion Sort, Quick Sort, Heap Sort e Radix Sort.

- **Bubble Sort:** Algoritmo iterativo baseado em trocas adjacentes, mantido no ecossistema de testes principalmente para fins de controle e contraste de desempenho assintótico.
- **Insertion Sort:** Método iterativo por inserção que, apesar de possuir complexidade quadrática no caso médio, foi escolhido por sua eficiência linear $O(n)$ em vetores pequenos e predominantemente já ordenados.
- **Quick Sort:** Algoritmo baseado no paradigma de divisão e conquista. Teve sua implementação aprimorada com o particionamento de Hoare e a estratégia de pivô por Mediana de Três.
- **Heap Sort:** Algoritmo baseado em estrutura de dados árvore. Foi integrado ao sistema devido à sua garantia teórica de tempo $O(n \log n)$ em qualquer cenário e por ser estritamente *in-place* ($O(1)$ de espaço), servindo como uma alternativa para grandes volumes de dados que possam degradar o Quick Sort.
- **Radix Sort (LSD):** Algoritmo de ordenação não-comparativa com tempo linear $O(n)$. Sua escolha justifica-se pela altíssima eficiência em cenários de dados massivos não-negativos, desde que a amplitude dos valores (*range*) seja proporcional ao tamanho do vetor.

A validação empírica dessas escolhas, bem como o mapeamento dos limites operacionais de cada algoritmo sob distribuições variadas, serão detalhados ao longo deste relatório.

Também foi feito um vídeo para apresentar o projeto, que pode ser visualizado no seguinte link:

www.nossoVideoDivo.com

2 Algoritmos Base

2.1 Bubble Sort

O Bubble Sort (Ordenação por Bolha) é um algoritmo de ordenação baseado no princípio de comparações e trocas adjacentes. Ele percorre o vetor repetidamente a partir do início comparando pares de elementos vizinhos (vetor[i] e vetor [i+1]), e troca-os de lugar se o elemento da esquerda for maior que o da direita. Esse processo é repetido até que nenhuma troca seja necessária em uma passada completa, indicando que o conjunto de dados está totalmente ordenado, sendo essa a versão aprimorada do Bubble Sort.

Dessa forma, a sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado. Nessa situação, o vetor só é percorrido uma única vez, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes devido ao crescimento quadrático das operações de comparação e movimentação de dados.

2.2 Insertion Sort

O Insertion Sort (Ordenação por Inserção) consiste em inserir um novo elemento em um vetor já ordenado ou quase ordenado. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição

para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é inserida. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é finalmente inserida.

Assim, sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado (ou quase ordenado). Nessa situação, a chave é comparada apenas uma vez com o elemento imediatamente anterior e o loop interno é interrompido por um break, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes e aleatórias.

2.3 Quick Sort

O algoritmo de ordenação Quick Sort é baseado no paradigma da divisão e conquista. Nesse sentido, a aplicação se consiste em ordenar o vetor em torno de um pivô escolhido, de modo que, os elementos à esquerda do pivô sejam menores que ele e os elementos à direita, consequentemente, sejam maiores. Dessa forma, o vetor é subdividido em duas partições, que terão o mesmo processo aplicado sobre elas, através de chamadas recursivas da função. Assim, as chamadas se encerram quando o caso base de um elemento na partição é atingido, uma vez que ele já estará ordenado.

Tais comportamentos destacados são os aspectos base do algoritmo quick sort, no entanto, cada aplicação dista da técnica utilizada para escolher o elemento pivô e para como o particionamento será realizado. Para o algoritmo implementado neste projeto, utilizou-se o método da mediana de 3 para se determinar o pivô. Ele se consiste em armazenar o primeiro, central e último elemento do vetor em variáveis auxiliares e retornar a mediana entre eles, nesse caso, o elemento do meio. Essa técnica, embora seja mais complexa do que aplicações comuns que utilizam o primeiro ou último elemento do array como pivôs, tem como benefícios a melhora do tempo de execução e a possibilidade da execução possuir uma complexidade de $O(n \log n)$ mesmo para vetores ordenados ou parcialmente ordenados - embora as métricas de análise dos vetores utilizadas para esse projeto evitem tais casos.

No caso da partição, existem três principais métodos comumente utilizados para o quick sort, o particionamento de Lomoto, Naive e Hoare, para este trabalho se utilizou o particionamento de Hoare. O método se consiste em na definição de dois ponteiros, nas duas extremidades do vetor e que, a cada interação, se movem em direção de seu encontro até que um elemento que necessita ser trocado de lugar seja encontrado. Considerando i o elemento na posição zero do vetor e j o elemento na posição final ambos são movidos até que i encontre um elemento maior que o pivô e, analogamente, j encontre um elemento menor que o pivô. Nesse caso, os valores encontrados trocam de posição e i e j voltam a percorrer o vetor até que eles se cruzem ou sobreponham. Destaca-se, que assim como a partição de Lomoto, o algoritmo de Hoare possui uma complexidade de tempo de $O(n)$ e de espaço de $O(1)$, contudo, ela é considerada mais rápida que o outro método, uma vez que, no geral, realiza menos trocas entre elementos. Em relação ao particionamento de Naive, o algoritmo se baseia na criação de um vetor auxiliar, ou seja, a complexidade de espaço é de $O(n)$, sendo significativamente menos eficiente que o método utilizado.

Dado as características da partição de Hoare, é importante destacar que o quick sort é um algoritmo *in-place*, ou seja, não utiliza nenhum vetor auxiliar para a ordenação, uma vez que as trocas são realizadas dentro do próprio vetor. Além disso, o algoritmo não é estável, uma vez que não preserva a ordem relativa entre elementos de mesmo valor. É também importante destacar que o quick sort apresenta um comportamento muito eficiente em termos de acesso à memória devido à forma como realiza o particionamento do vetor. Durante essa etapa, os elementos são percorridos sequencialmente por meio de índices que avançam a partir das extremidades do arranjo, favorecendo a localidade espacial dos dados. Como elementos próximos na memória tendem a ser acessados em instantes próximos, a hierarquia de memória do processador, especialmente os níveis de cache, é utilizada de maneira mais eficiente. Além disso, à medida que as partições são realizadas, o algoritmo passa a operar sobre subvetores progressivamente menores, os quais frequentemente passam a caber integralmente na memória cache. Dessa forma, reduz-se a quantidade de acessos à memória principal, resultando em uma diminuição da latência de acesso aos dados e contribuindo significativamente para o elevado desempenho prático do algoritmo.

Ademais, é possível analisar o melhor, pior e médio caso do quick sort. Nota-se que o melhor caso ocorre quando o pivô selecionado é exatamente o elemento central do vetor analisado.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Pela análise da recursão, através do Teorema Mestre, obtem-se a complexidade de espaço para o melhor caso como $O(n \log n)$. Nesse sentido, para o médio caso, considera-se que o array é dividido em duas partes de tamanho K , sendo assim:

$$T(n) = T(n - K) + T(n)$$

Após a resolução da recorrência, também nota-se que a complexidade é de $O(n \log n)$. Nesse sentido, evidencia-se que no melhor e médio caso, o quick sort utiliza em média $2 n \log n$ comparações entre elementos distintos do vetor para ordená-lo, e aproximadamente, um sexto desse número de trocas. Por fim, o pior caso ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do subvetor analisado. Nessa situação, a cada etapa do algoritmo são geradas partições extremamente desbalanceadas, contendo 0 e $n - 1$ elementos, o que resulta em uma árvore de recursão linear e leva a uma complexidade temporal de $O(n^2)$:

$$T(n) = T(n - 1) + n$$

Para o pior caso, observa-se que o algoritmo necessita realizar, em média, $n^2/2$ comparações para ordenar o vetor, o que representa um aumento assintótico significativo em relação aos outros casos. Em relação à complexidade de espaço, cada chamada recursiva apresenta complexidade $O(1)$ - devido a característica *in-place* do algoritmo -, no entanto, múltiplas chamadas ocorrem simultaneamente, modificando o comportamento da complexidade espacial conforme o caso de execução. Para o médio e melhor caso, a árvore de recursão gerada é balanceada e possui profundidade de $O(\log n)$, o que acarreta em uma complexidade de espaço de $O(\log n)$. Em contrapartida, devido ao comportamento linear da árvore no pior caso, sua altura é de $O(n)$, acarretando em uma complexidade assintótica de $O(n)$. Assim, foi possível elaborar a seguinte tabela comparativa:

Variação	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(n \log n)$	$O(\log n)$
Caso médio	$O(n \log n)$	$O(\log n)$
Pior caso	$O(n^2)$	$O(n)$

Tabela 1: Complexidade de tempo e espaço do quick sort.

2.4 Heap Sort

O Heap Sort baseia-se na estrutura de dados *Max-Heap*, uma árvore binária quase completa mapeada implicitamente em um vetor. Para um nó no índice i , seus filhos estão localizados em $2i + 1$ e $2i + 2$, e o valor do pai é sempre maior ou igual ao de seus filhos.

A ordenação ocorre em duas etapas: a construção do heap (**Build-Max-Heap**), que reorganiza o vetor em tempo linear, e a extração sucessiva do elemento máximo. A cada extração, o último elemento do subvetor substitui a raiz, e a função **Max-Heapify** restaura a propriedade da árvore.

Matematicamente, a etapa de construção possui custo assintótico linear:

$$O\left(\sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}}\right) = O(n)$$

A segunda etapa percorre a altura da árvore para $n - 1$ extrações, custando $O(\log_2 n)$ cada. Assim, a complexidade total resulta em:

$$T(n) = O(n) + O(n \log_2 n) = O(n \log_2 n)$$

Isso garante ao Heap Sort um desempenho de $O(n \log_2 n)$ em qualquer cenário, blindando-o contra piores casos. Além disso, é um algoritmo estritamente *in-place*, com complexidade espacial de $O(1)$.

Apesar da estabilidade matemática, o Heap Sort sofre em hardwares modernos devido à sua péssima localidade espacial de referência. A navegação não-sequencial pelos índices da árvore (i para $2i + 1$ e

$2i + 2$) esgota rapidamente a capacidade da memória Cache (L2/L3). Em vetores massivos, isso gera uma alta taxa de *Cache Misses*, forçando a CPU a paralisar seus ciclos para buscar dados na memória RAM, o que degrada o seu desempenho prático.

Variação	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(n \log_2 n)$	$O(1)$
Caso médio	$O(n \log_2 n)$	$O(1)$
Pior caso	$O(n \log_2 n)$	$O(1)$

Tabela 2: Complexidade de tempo e espaço do Heap Sort.

2.5 Radix Sort

O Radix Sort é um algoritmo não-comparativo que supera o limite estrutural de $\Omega(n \log_2 n)$ operando sobre os dígitos individuais dos elementos. Em sua variante *Least Significant Digit* (LSD), o processamento ocorre da direita para a esquerda (das unidades para as casas maiores). Para isso, o método exige obrigatoriamente um algoritmo auxiliar estável, tipicamente o Counting Sort.

A aplicação identifica o maior elemento para determinar o número máximo de dígitos (d). Em seguida, o Counting Sort é invocado d vezes para agrupar e reordenar os elementos em cada casa decimal. Sendo n o tamanho do vetor e k a base numérica, cada rodada custa $O(n + k)$, resultando na complexidade de tempo:

$$T(n) = O(d \cdot (n + k))$$

Para tipos de dados nativos onde d e k são tratados como constantes, o comportamento assintótico simplifica-se para um modelo linear de $O(n)$.

Entretanto, essa eficiência temporal cobra um alto preço em hardware. O Radix Sort não é *in-place*, exigindo a alocação de vetores auxiliares com complexidade espacial de $O(n + k)$. Em conjuntos de dados massivos, essa alocação pode exaurir rapidamente a memória RAM do sistema, forçando a paginação no disco de armazenamento (HD/SSD) e elevando a latência de forma drástica. Adicionalmente, o tratamento de números negativos exige rotinas extras de *offsetting* (deslocamento numérico unificado) para evitar falhas de acesso, adicionando sobrecarga ao algoritmo inicial.

Variação	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(d \cdot (n + k))$	$O(n + k)$
Caso médio	$O(d \cdot (n + k))$	$O(n + k)$
Pior caso	$O(d \cdot (n + k))$	$O(n + k)$

Tabela 3: Complexidade de tempo e espaço do Radix Sort.

3 Arquitetura do Sistema

3.1 Execução da Aplicação

A aplicação foi desenvolvida para ser executada por meio da linha de comando, permitindo ao usuário configurar diferentes parâmetros relacionados à geração dos dados de entrada e ao algoritmo de ordenação utilizado. A interface foi projetada de forma a facilitar a realização de experimentos em larga escala, possibilitando a automatização dos testes e a coleta sistemática das métricas de desempenho.

A sintaxe geral de execução do programa é dada por:

```
./programa [opções]
```

Em que as opções disponíveis permitem selecionar o algoritmo de ordenação, definir a origem dos dados de entrada e especificar características da distribuição dos valores.

3.1.1 Seleção do Algoritmo

O algoritmo de ordenação pode ser escolhido por meio da flag `--algoritmo`. Atualmente, a aplicação suporta os algoritmos Heap Sort, Quick Sort, Insertion Sort, Bubble Sort e Radix Sort.

Por exemplo, para executar o Quick Sort sobre um vetor gerado dinamicamente com 100.000 elementos:

```
./programa --algoritmo quick --tamanho 100000
```

Também é possível utilizar o modo automático:

```
./programa --algoritmo auto --tamanho 100000
```

Nesse caso, o sistema realiza uma caracterização prévia do vetor e seleciona automaticamente o algoritmo considerado mais adequado para aquela entrada.

3.1.2 Geração Dinâmica de Vetores

A geração dinâmica é realizada através da flag `--tamanho`, que define a quantidade de elementos do vetor.

```
./programa --algoritmo quick --tamanho 1000000
```

Além do tamanho, o usuário pode especificar a distribuição dos dados utilizando a flag `--distribuicao`. Por exemplo:

```
./programa --algoritmo quick --tamanho 1000000 --distribuicao aleatorio
```

As distribuições atualmente disponíveis são:

- aleatorio
- quase_ordenado
- crescente
- decrescente
- discrepante
- organ_pipe
- intercalado

Por exemplo, para gerar um vetor utilizando a distribuição crescente:

```
./programa --algoritmo quick --tamanho 1000000 --distribuicao crescente
```

3.1.3 Controle do Grau de Desordem

Para vetores quase ordenados, o usuário pode especificar o percentual de elementos que sofrerão trocas aleatórias através da flag `--desordem`.

Por exemplo, para gerar um vetor com apenas 5% de desordem:

```
./programa --algoritmo quick --tamanho 1000000 --distribuicao quase_ordenado --desordem 5
```

Da mesma forma, pode-se aumentar o grau de desordem:

3.1.4 Leitura de Dados a Partir de Arquivos

A aplicação também permite utilizar vetores previamente armazenados em arquivos texto. Para isso, utiliza-se a flag `--input` seguida do caminho do arquivo.

```
./programa --algoritmo heap --input dados.txt
```

Assim, o programa realiza automaticamente a leitura dos valores contidos no arquivo, determina o tamanho da entrada e executa o algoritmo selecionado. Observa-se que o programa contabiliza todos os diferentes valores do arquivo como um único vetor.

3.1.5 Inserção Manual de Dados

Para testes rápidos e depuração, a aplicação permite a inserção direta dos elementos do vetor pela linha de comando utilizando a flag `--array`.

Por exemplo:

```
./programa --algoritmo quick --array 8 3 1 7 5 2 9 4
```

Nesse caso, o vetor utilizado pelo algoritmo será:

[8 3 1 7 5 2 9 4]

3.1.6 Saída do Programa

Ao final da execução, a aplicação apresenta no terminal uma tabela contendo as principais métricas coletadas durante a ordenação, incluindo: Além da exibição em tela, todos os resultados são armazenados automaticamente em um arquivo CSV localizado no diretório `output`, permitindo posterior análise estatística e construção de gráficos.

3.2 Distribuições dos Vetores Utilizados

Com o objetivo de avaliar o comportamento dos algoritmos de ordenação sob diferentes condições de entrada, foram implementadas diversas estratégias de geração de vetores. Cada distribuição busca representar cenários específicos, permitindo analisar tanto o desempenho e a eficiência dos algoritmos diante de diferentes padrões de organização dos dados.

3.2.1 Vetor Aleatório

Nesta distribuição, cada posição do vetor recebe um valor pseudoaleatório gerado independentemente das demais posições. Os valores são obtidos por meio de uma distribuição uniforme em um intervalo previamente definido.

Para um vetor de tamanho (n), um exemplo de instância gerada é:

$A = [42 \ 17 \ 91 \ 3 \ 58 \ 24 \ \dots]$

Essa distribuição representa o caso mais comum em análises experimentais e é frequentemente utilizada como referência para comparação entre algoritmos de ordenação.

3.2.2 Vetor Quase Ordenado

A distribuição quase ordenada é construída inicialmente em ordem crescente. Em seguida, uma porcentagem previamente definida dos elementos tem suas posições trocadas aleatoriamente.

Para um vetor de tamanho (n), um exemplo é:

$A = [1 \ 2 \ 3 \ 4 \ 8 \ 6 \ 7 \ 5 \ 9 \ 10 \ \dots \ n]$

Essa distribuição permite avaliar a capacidade dos algoritmos de explorar a existência de ordenação prévia nos dados.

3.2.3 Vetor Discrepante

A distribuição discrepante é composta majoritariamente por valores pertencentes a um intervalo reduzido. Posteriormente, uma pequena fração dos elementos é substituída por valores extremamente elevados, denominados *outliers*.

Para um vetor de tamanho (n), um exemplo é:

$$A = [15 \ 23 \ 8 \ 41 \ 10^9 \ 12 \ 4 \ 37 \ \dots]$$

Essa configuração permite analisar o impacto da presença de valores discrepantes no desempenho dos algoritmos de ordenação.

3.2.4 Vetor Crescente

Nesta distribuição, os elementos são armazenados em ordem estritamente crescente.

Para um vetor de tamanho (n), a sequência gerada é dada por:

$$A = [1 \ 2 \ 3 \ \dots \ n-2 \ n-1 \ n]$$

Trata-se de um caso clássico utilizado na análise de algoritmos de ordenação, uma vez que determinadas estratégias de seleção podem apresentar comportamentos significativamente distintos quando os dados já se encontram ordenados.

3.2.5 Vetor Decrescente

Nesta distribuição, os elementos são armazenados em ordem estritamente decrescente.

Para um vetor de tamanho (n), a sequência gerada é dada por:

$$A = [n \ n-1 \ n-2 \ \dots \ 3 \ 2 \ 1]$$

Assim como o vetor crescente, essa distribuição é amplamente utilizada na literatura para avaliar a sensibilidade dos algoritmos à disposição inicial dos dados.

3.2.6 Distribuição Organ Pipe

A distribuição *organ pipe* apresenta valores crescentes até uma posição central e, a partir desse ponto, valores decrescentes, formando uma estrutura semelhante aos tubos de um órgão musical, o que origina o seu nome.

De forma geral, para um vetor de tamanho ímpar (n), a sequência possui a forma:

$$A = \left[1 \ 2 \ 3 \ \dots \ \frac{n+1}{2} \ \dots \ 3 \ 2 \ 1 \right]$$

Por exemplo, para ($n=9$):

$$A = [1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1]$$

3.2.7 Distribuição Intercalada

A distribuição intercalada alterna elementos de pequeno e grande valor ao longo de todo o vetor. De forma geral, para um vetor de tamanho (n), a sequência é construída segundo o padrão:

$$A = [1 \ n \ 2 \ n-1 \ 3 \ n-2 \ \dots]$$

Por exemplo, para ($n=10$):

$$A = [1 \ 10 \ 2 \ 9 \ 3 \ 8 \ 4 \ 7 \ 5 \ 6]$$

Observa-se que as distribuições *organ pipe* e intercalada foram incluídas especificamente para investigar possíveis degradações de desempenho do quick sort utilizando a estratégia da mediana de três para seleção do pivô, que será especificado na devida sessão de resultados.

3.3 Dados de Desempenho Coletados

Diversos dados foram coletados durante as execuções e testes dos algoritmos, com o intuito de estudar sistematicamente o comportamento de cada método no cenários analisados. Nesse sentido, os dados de desempenho coletados foram:

- **Tempo de execução:** determinação do tempo de execução da ordenação em segundos;
- **Número de comparações:** número total de comparações entre elementos do vetor durante a execução;
- **Número de trocas:** mede a quantidade de trocas de valores entre dois elementos do vetor, nos códigos implementados, a função de troca foi modularizada e chamada por todos os algoritmos conforme a necessidade;
- **Número de operações:** quantifica a quantidade de operações totais realizada pelo algoritmo durante a execução - como atribuição de valores, operações matemáticas, comparações e outros-;
- **Número de cópias:** determina a quantidade de vezes que cópias de elementos e vetores são realizadas;
- **Chamadas recursivas:** quantifica o total de chamadas recursivas necessárias para a ordenação;
- **Profundidade Máxima da Recursão:** mede a altura máxima da árvore de recursão percorrida pelo algoritmo, ou de forma equivalente, O maior número de chamadas da função recursiva simultaneamente ativas na pilha de execução durante toda a ordenação;
- **Memória Extra Utilizada:** determina a quantidade de memória auxiliar utilizada para a ordenação através de uma estimativa envolvendo as variáveis auxiliares criadas e o tamanho delas em bytes.

Observa-se que nem todos os dados foram coletados para todos os algoritmos, uma vez que eles diferem da arquitetura de implementação - iterativo ou recursivo - e de particularidades mais intrínsecas. Dessa forma, algumas métricas podem não condizer com alguns algoritmos.

3.4 Métricas Coletadas

O sistema adaptativo desenvolvido tem como objetivo selecionar automaticamente o algoritmo de ordenação mais adequado para cada vetor de entrada. Para isso, antes da ordenação, são coletadas diversas métricas que descrevem as características dos dados. A partir dessas informações, um conjunto de heurísticas decide qual algoritmo apresenta maior potencial de desempenho para a entrada analisada.

3.4.1 Grau de Desordem

O grau de desordem mede o percentual de inversões adjacentes existentes no vetor. Uma inversão adjacente ocorre quando um elemento é maior que seu sucessor imediato.

A métrica é calculada por:

$$Desordem = \frac{\text{Quantidade de Inversões Adjacentes}}{n - 1} \times 100$$

Essa métrica foi utilizada para identificar vetores quase ordenados, situação na qual o Insertion Sort apresenta desempenho próximo de $O(n)$.

3.4.2 Quantidade de Elementos Repetidos

Esta métrica contabiliza quantos elementos do vetor são cópias extras de valores já existentes.

Seu objetivo inicial era investigar se a presença de muitos valores repetidos poderia influenciar a escolha do algoritmo de ordenação.

3.4.3 Amplitude dos Valores (Range)

A amplitude dos valores corresponde à diferença entre o maior e o menor elemento do vetor:

$$Range = Max - Min$$

Essa métrica foi utilizada para avaliar a adequação do vetor à aplicação do Radix Sort.

3.4.4 Desvio Padrão

O desvio padrão mede o grau de dispersão dos elementos em relação à média do conjunto de dados.

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

Inicialmente investigou-se a possibilidade de utilizar essa métrica como parâmetro para seleção do algoritmo.

3.4.5 Valores Máximo e Mínimo

Além de serem utilizados para o cálculo da amplitude dos dados, os valores máximo e mínimo permitem identificar a presença de números negativos, informação importante para a escolha do algoritmo de ordenação.

3.5 Seleção das Métricas

Embora todas as métricas tenham sido implementadas e avaliadas experimentalmente, apenas aquelas que demonstraram influência consistente no desempenho dos algoritmos foram incorporadas à heurística final do sistema.

Métrica	Utilizada na heurística
Grau de desordem	Sim
Quantidade de repetidos	Não
Amplitude (Range)	Sim
Desvio padrão	Não
Valor mínimo	Sim

Tabela 4: Métricas implementadas e utilização na heurística final

A quantidade de elementos repetidos e o desvio padrão foram mantidos apenas para fins de caracterização dos conjuntos de dados. Durante os experimentos realizados, essas métricas não apresentaram influência significativa na escolha do algoritmo mais eficiente.

3.6 Heurística Final

Após a realização dos experimentos, a seguinte estratégia foi adotada:

```
if (n <= 1000 && desordem <= 5.0)
    return "insertion";

if (min < 0 && n >= 10000)
    return "heap";

if (min >= 0 && range <= 10*n)
    return "radix";

return "quick";
```

A lógica empregada pode ser resumida da seguinte forma:

- Vetores pequenos e quase ordenados são processados pelo Insertion Sort.
- Vetores contendo valores negativos podem ser direcionados ao Heap Sort.
- Vetores cuja amplitude seja proporcional ao tamanho são direcionados ao Radix Sort.
- Os demais casos são tratados pelo Quick Sort.

A regra associada à presença de valores negativos foi incluída por razões teóricas relacionadas à aplicabilidade do Radix Sort.

Os conjuntos de dados utilizados durante os experimentos continham apenas valores não negativos, impossibilitando a validação experimental dessa decisão. Dessa forma, a utilização do Heap Sort para vetores negativos de grande porte foi baseada em propriedades teóricas dos algoritmos analisados e deve ser entendida como uma decisão conservadora da heurística.

4 Validação das Heurísticas

A definição da heurística final foi baseada em uma série de hipóteses experimentais avaliadas durante o desenvolvimento do projeto.

4.1 Hipótese 1: Vetores Pequenos Favorecem o Insertion Sort

Inicialmente assumiu-se que vetores com menos de 50 elementos deveriam ser processados pelo Insertion Sort.

Para validar essa hipótese, foram executados experimentos utilizando vetores aleatórios de tamanho 50 e comparando os resultados obtidos por todos os algoritmos implementados.

Os resultados mostraram que o Quick Sort apresentou desempenho superior ao Insertion Sort mesmo para esse tamanho de entrada.

Dessa forma, a regra:

```
if (n <= 50)
    return "insertion";
```

foi removida da heurística final.

Conclusão: hipótese rejeitada.

4.2 Hipótese 2: Vetores Quase Ordenados Favorecem o Insertion Sort

Foram realizados experimentos com vetores quase ordenados contendo níveis crescentes de desordem.

Observou-se que o Insertion Sort apresentou desempenho significativamente superior para vetores com grau de desordem reduzido, mantendo comportamento próximo ao caso linear esperado pela teoria.

A heurística baseada em:

```
n <= 1000 && desordem <= 5
```

mostrou-se consistente durante todos os experimentos realizados.

Conclusão: hipótese confirmada.

É importante destacar que vetores ordenados ou quase invertidos normalmente representam entradas problemáticas para implementações simples do Quick Sort que utilizam o primeiro ou o último elemento como pivô. Entretanto, a implementação desenvolvida neste trabalho utiliza a estratégia da mediana de três para seleção do pivô, considerando os elementos inicial, central e final do subvetor.

Essa abordagem reduz significativamente a probabilidade de partições extremamente desbalanceadas em vetores parcialmente ordenados ou invertidos. Como consequência, observou-se experimentalmente que o Quick Sort manteve desempenho compatível com o comportamento esperado de $O(n \log n)$ nesses cenários.

4.3 Hipótese 3: O Radix Sort Necessita de um Tamanho Mínimo

Inicialmente foi considerada a utilização de um limite inferior para ativação do Radix Sort.

Entretanto, testes realizados com vetores de aproximadamente 5000 elementos mostraram que o algoritmo já apresentava desempenho superior ao Quick Sort.

Dessa forma, o limite mínimo inicialmente considerado foi removido.

Conclusão: hipótese rejeitada.

4.4 Hipótese 4: Vetores Grandes Favorecem o Radix Sort

Experimentos realizados com vetores aleatórios de 10000 elementos confirmaram que o Radix Sort apresenta desempenho superior quando a amplitude dos valores é compatível com o tamanho do vetor.

Conclusão: hipótese confirmada.

4.5 Hipótese 5: Vetores Discrepantes Devem Evitar o Radix Sort

Considerou-se inicialmente que vetores com amplitudes extremamente elevadas deveriam evitar o uso do Radix Sort.

Os resultados mostraram que, mesmo em cenários discrepantes, o Radix Sort continuou apresentando desempenho competitivo. Entretanto, observou-se que o aumento da amplitude dos dados pode tornar o Quick Sort uma alternativa interessante devido ao menor consumo de memória auxiliar.

Dessa forma, conclui-se que a amplitude dos dados deve ser considerada pela heurística, porém não foi observada uma degradação suficientemente significativa que justificasse a exclusão completa do Radix Sort nesses cenários.

Conclusão: hipótese parcialmente aceita.

4.6 Hipótese 6: Vetores Muito Grandes Favorecem o Heap Sort

Também foi investigada a possibilidade de utilizar o Heap Sort para vetores extremamente grandes.

Foram realizados experimentos com vetores contendo até 500000 elementos.

Os resultados mostraram que o Heap Sort permaneceu consistentemente inferior ao Quick Sort e ao Radix Sort em termos de tempo de execução e número total de operações.

Consequentemente, não foi identificada uma situação experimental em que sua utilização apresentasse vantagens claras.

Conclusão: hipótese rejeitada.

5 Metodologia Experimental

5.1 Ambiente de Execução e Ferramentas

A respeito da realização o projeto, foi utilizado como ambiente de desenvolvimento (IDE), o Visual Studio Code (VSCode), para a escrita, depuração e organização modular do código em C. Para o controle de versão e a colaboração entre os membros do grupo, utilizou-se o Git para o versionamento local do código-fonte e o GitHub como plataforma de hospedagem e sincronização do repositório remoto.

5.2 Conjuntos de Dados (Datasets) e Distribuições

Com base na estratégia de validação do script de *benchmark*, os algoritmos foram submetidos a cenários específicos projetados para confrontar a heurística de decisão automática contra execuções estáticas individuais. Foram utilizadas três distribuições principais de dados:

- **Quase Ordenado:** Vetores com baixo grau de desordem (inversões adjacentes limitadas a até 5%);
- **Aleatório:** Valores pseudoaleatórios distribuídos uniformemente;
- **Decrescente:** Configuração de pior caso linear onde os elementos estão em ordem estritamente inversa.

5.3 Volumetria e Tamanhos Testados

A volumetria dos testes foi dividida em quatro fases assintóticas no script de automação para avaliar os limiares de transição do sistema adaptativo:

- **Pequeno Porte (Foco em *Insertion Sort*):** Arranjos de 100, 500 e 1.000 elementos na distribuição quase ordenada, incluindo um limiar de 2.000 para forçar a alternância do sistema ao estourar a eficiência do algoritmo;
- **Médio/Grande Porte (Foco em *Heap Sort*):** Vetores de 10.000, 50.000 e 100.000 elementos para validar o acionamento de segurança do algoritmo;
- **Grande Porte (Foco em *Radix Sort*):** Volumes de 500.000, 1.000.000 e 5.000.000 de elementos em distribuição aleatória, onde os métodos não-comparativos superam o limite inferior dos baseados em comparação;
- **Altíssimo Porte (Foco em *Quick Sort*):** Testes extremos com 2.000.000, 4.000.000 e 8.000.000 de elementos na distribuição decrescente para validar o algoritmo como a resposta padrão (*fallback*) do sistema.

6 Resultados

6.1 Bubble Sort

Ao implementar o Bubble Sort, foi utilizada uma flag booleana (`houveTroca`) para identificar se o vetor está ordenado após percorrê-lo. Assim, essa estratégia corroborou a eficiência do algoritmo diante de entradas já ordenadas. Diante de diversas entradas com diferentes características, foi possível organizar as informações na Tabela 5.

Tamanho (n)	Cenário de entrada	Tempo de execução (s)	Comparações	Trocas
1000	Já Ordenado (Melhor caso)	0.000	999	0
1000	Aleatório	0.003	498870	240479
1000	Invertido (Pior caso)	0.002	499500	499500

Tabela 5: Tabelas e Métricas coletadas - BubbleSort.

A fim de visualizar melhor a complexidade temporal do Bubble Sort nos três casos distintos (melhor caso, caso médio e pior caso), foram construídos gráficos em função do tamanho N do vetor e do tempo de execução.

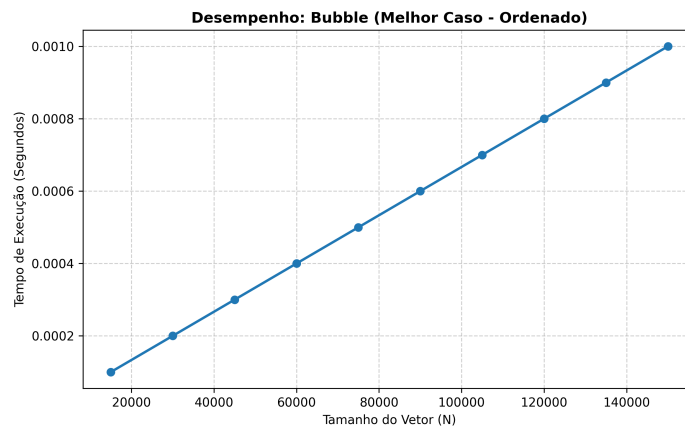


Figura 1: Desempenho Bubble Sort: Melhor caso

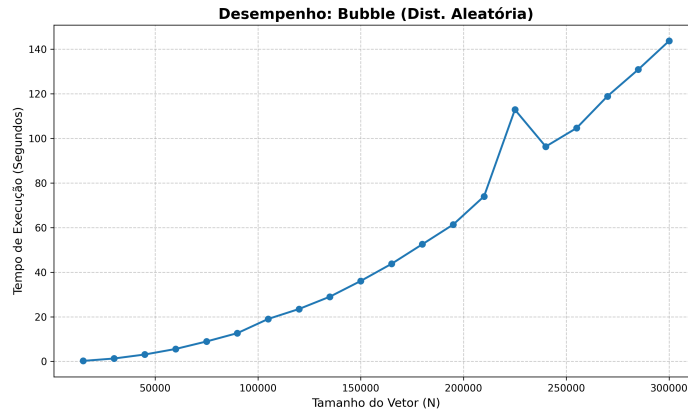


Figura 2: Desempenho Bubble Sort: Caso médio

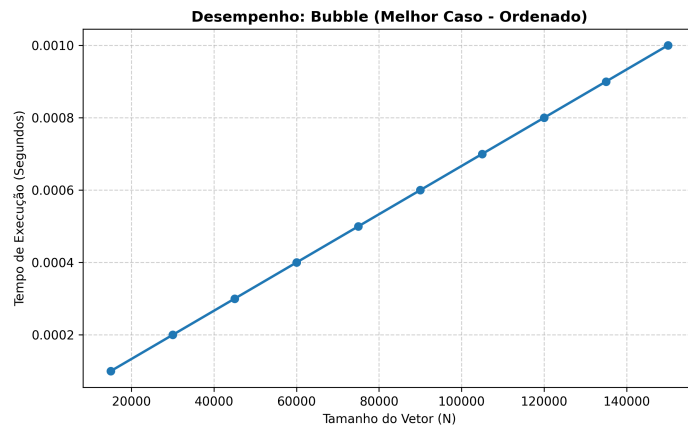


Figura 3: Desempenho Bubble Sort: Pior caso

Analisando os três gráficos gerados, observa-se que o algoritmo se comporta da maneira esperada para as entradas recebidas, possuindo uma complexidade temporal quadrática $O(n^2)$ no caso médio (entradas aleatórias) e pior caso, e complexidade temporal $O(n)$ no melhor caso (vetor ordenado). No gráfico da Figura 2, que exibe o comportamento no caso médio, é possível observar um pico inesperado no tempo de execução para o vetor de tamanho 225.000. Como o número de comparações e trocas matemáticas manteve o crescimento estritamente quadrático correspondente à teoria assintótica, conclui-se que essa variação não possui relação com a lógica interna do Bubble Sort, e sim de uma interferência ambiental externa relacionada ao gerenciamento de processos do Sistema Operacional. Processos que rodam em segundo plano disputam o uso do processador, diminuindo temporariamente o tempo de execução dedicado ao programa.

A respeito do impacto das características da entrada, nota-se que o comportamento do Bubble Sort é fortemente governado pelo estado inicial do vetor. O algoritmo responde de forma excelente a vetores já ordenados devido à otimização por flag implementada, reduzindo o custo computacional de quadrático para linear. Contudo, em cenários onde a desordem é moderada ou total (aleatório), o algoritmo perde eficiência rapidamente, pois o número de passadas necessárias para arrastar os elementos até suas posições corretas cresce massivamente.

Nesse tipo de algoritmo, a entrada adversarial para o Bubble Sort ocorre quando o vetor está perfeitamente invertido (ordem decrescente). Neste cenário, o elemento que deveria estar na última posição está na primeira, forçando o algoritmo a realizar o limite máximo teórico de comparações e trocas através da fórmula $\frac{n(n-1)}{2}$. Para grandes volumes de dados ($n > 10.000$), essa quantidade de operações sobrecarrega o processador, tornando o algoritmo lento demais para ser utilizado.

6.2 Insertion Sort

Para analisar o comportamento do Insertion Sort e validar as complexidades temporais esperadas, foi organizada a Tabela 6 a partir de entradas com características distintas.

Tamanho (n)	Cenário de entrada	Tempo de execução	Comparações	Trocas
1000	Já Ordenado (Melhor caso)	0.000	999	0
1000	Aleatório	0.000	254782	253791
1000	Invertido (Pior caso)	0.002	499500	499500

Tabela 6: Tabelas e Métricas coletadas - Insertion Sort.

Pela tabela, observa-se a mesma tendência de comportamento que o Bubble Sort: complexidade temporal quadrática $O(n^2)$ no caso médio e pior caso, e complexidade temporal $O(n)$ no melhor caso (vetor ordenado ou quase ordenado). O comportamento do Insertion sort para entradas aleatórias pode ser visualizado no gráfico presente na Figura 4.

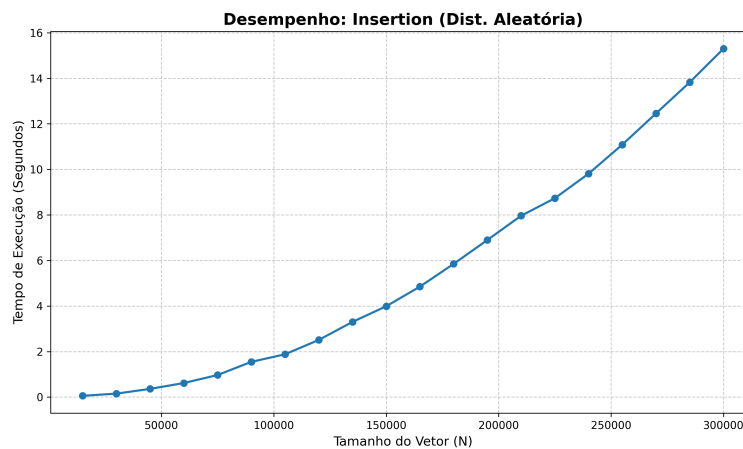


Figura 4: Desempenho Insertion Sort: distribuição aleatória

Pela Figura 4, para uma distribuição aleatória do vetor, a complexidade de tempo do algoritmo é $O(n^2)$, conforme previsto para o caso médio. O Insertion Sort demonstra uma sensibilidade extrema ao estado inicial do vetor. O algoritmo responde de forma quase linear e altamente eficiente quando a entrada já está estruturada de maneira favorável (ordenada ou quase ordenada).

Diferente de algoritmos como o Bubble Sort tradicional, que realiza checagens redundantes, o Insertion Sort consegue consolidar faixas ordenadas à esquerda rapidamente, diminuindo drasticamente o tempo de execução do código. A pior entrada possível (adversarial) para o Insertion Sort ocorre quando o vetor está em ordem estritamente decrescente. Neste cenário, cada novo elemento analisado como "chave" é menor do que todos os números que já foram organizados à sua esquerda, forçando o algoritmo a realizar o deslocamento máximo de dados de todas as posições anteriores a cada rodada.

Um comportamento que pode parecer contra-intuitivo nos testes práticos é o fato de o Insertion Sort apresentar um tempo de execução e número de operações significativamente menores do que o Bubble Sort em cenários aleatórios, mesmo ambos compartilhando a mesma complexidade teórica $O(n^2)$. Isso acontece porque o Insertion Sort interrompe sua busca interna assim que encontra o lugar correto do elemento, o que estatisticamente reduz os deslocamentos pela metade. Já o Bubble Sort não possui essa parada imediata no laço interno e continua varrendo o restante das posições adjacentes, gerando mais desperdício de processamento.

6.3 Quick Sort

Com o intuito de validar a algoritmo quick sort implementado neste projeto, analisou-se as curvas do número de comparações entre elementos do vetor e número de trocas necessárias para atingir a ordenação - para valores com diferentes ordens de grandeza - quais foram estudadas teoricamente e

possuem comportamento esperado definido. Dessa forma, nota-se que o número de comparações do quick sort para o melhor caso é na ordem de $2 n \log n$, e um sexto desse número para a quantidade de trocas.

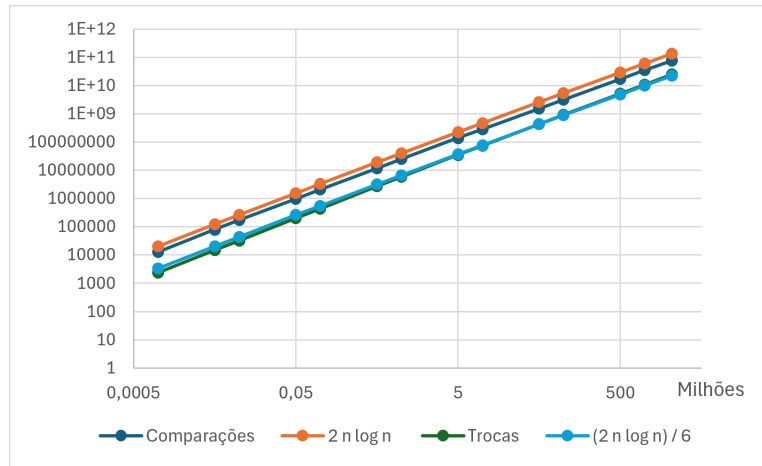


Figura 5: Comparativo entre o número de comparações e trocas e seu comportamento esperado para o melhor caso do quick sort. Escala dílog.

No gráfico é possível observar que o comportamento esperado é efetivo, uma vez que os dados coletados acompanham as linhas das funções que descrevem o comportamento previsto. Observa-se que o gráfico foi elaborado em escala log-log para a melhor visualização do comportamento para as diferentes ordens de grandeza.

Na implementação do quick sort desenvolvida, o pivô é escolhido por meio do método da mediana de três, utilizando os elementos inicial, central e final do subvetor. Essa estratégia reduz significativamente a ocorrência de partições desbalanceadas, especialmente em vetores já ordenados ou parcialmente ordenados. Ainda assim, o pior caso permanece teoricamente possível e ocorre quando a mediana desses três elementos produz repetidamente pivôs próximos aos extremos do subvetor, gerando partições de tamanhos aproximadamente 0 e $n - 1$. Nesse sentido, o método da mediana de três foi amplamente estudada por cientistas como David Musser, pesquisador conhecido por desenvolver o algoritmo Introsort, atualmente utilizado em diversas bibliotecas padrão da linguagem C++. Em seu trabalho, Musser demonstrou que, embora a estratégia da mediana de três reduza significativamente a probabilidade de partições desbalanceadas em entradas comuns, ela não elimina completamente o pior caso do Quick Sort. Sob essa ótica, existem famílias de sequências especialmente construídas, conhecidas como *Median-of-Three Killer Sequences*, capazes de induzir a escolha repetida de pivôs inadequados, resultando em partições altamente desbalanceadas e levando o algoritmo a uma complexidade temporal quadrática $O(n^2)$.

Algumas dessas sequências conhecidas são a *organ pipe* e intercalada. A sequência *organ pipe* se consiste em um array com o elemento central possuindo o maior valor do vetor e os elementos a esquerda e a direita dele decrescendo. Um exemplo de uma distribuição dessa forma, contínua, para $n = 10$ é: $[1\ 2\ 3\ 4\ 5\ 6\ 4\ 3\ 2\ 1]$. Em relação a distribuição intercalada, o vetor se consiste em valores intercalados entre uma sequência crescente e outra decrescente, dessa forma, para $n = 10$, uma possível configuração seria $[1\ 10\ 2\ 9\ 3\ 8\ 4\ 7\ 5\ 6]$. No gráfico a seguir, é possível visualizar um comparativo entre os tempos de execução para vetores de distribuição aleatória e distribuição seguindo o padrão *organ pipe*:

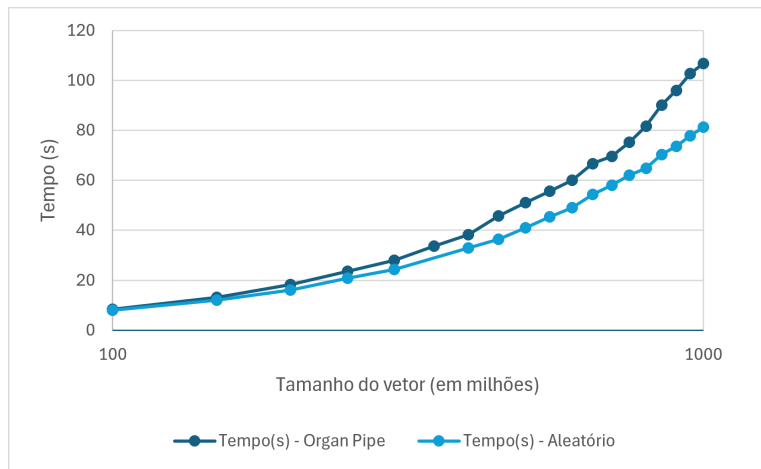


Figura 6: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Através dos dados, é possível observar que essa distribuição não estressou devidamente o algoritmo, uma vez que a curva de ambos apresentou uma complexidade temporal de $O(n \log n)$, embora a dsitribuição *organ pipe* tenha apresentado tempos de execução ligeiramente maiores. É válido destacar o comportamento da métrica que mede a profundidade máxima árvore de rescursão para ambos os casos.

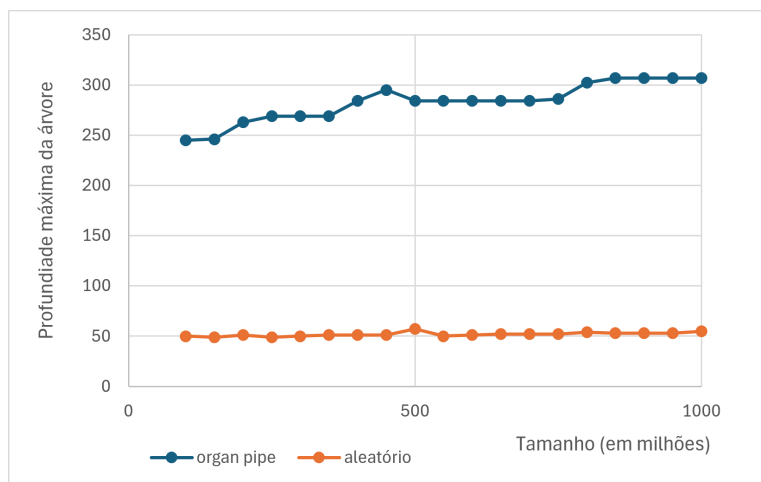


Figura 7: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Observa-se que a profundidade máxima para a distribuição *organ pipe* foi significativamente maior do que para o caso aleatório, indicando que a escolha do pivô para esses vetores não foi tão eficaz quanto o observado no melhor caso, o que acarretou na necessidade de um maior número de partições e, consequentemente, uma maior árvore recursiva. Assim, os arrays criandos seguindo o modelo *organ pipe* não estressaram o algoritmo implementado o suficiente para prejudicar a sua complexidade assintótica de tempo para $O(n^2)$, à medida que, como observado e analisado, a ditribuição se enquadra em um médio caso do quick sort.

Em relação aos vetores intercalados, o quick sort não conseguiu ordenar arrays com mais de 100 milhões de tamanho, uma vez que a péssima escolha dos pivôs para esse caso acarretaram em árvores de recursão extremamente grandes - com profundidade máxima maior que 80.000 para vetores na ordem de 200 milhões de tamanho - o que ocasionou a interrupção prematura da ordenação devido ao esgotamento da pilha de chamadas. Dessa forma, observa-se que o pior caso esperado foi atingido, acarretando em uma execução na ordem de $O(n^2)$ e incapaz de ser finalizada para vetores grandes,

como os analisados. Nesse sentido, evidencia-se também que para vetores menores aos analisados, de ordem de grandeza 10.000, a profundidade máxima da recursão atingiu valores muito maiores do que a ordenação de vetores aleatórios na casa de 100 milhões de tamanho - por exemplo, um vetor aleatório de tamanho 1 bilhão atingiu profundidade máxima de 55, enquanto um vetor intercalado de 100 mil, atingiu 507. Esses comportamentos destacam que a estratégia da mediana de três, embora eficiente em casos usuais, permanece suscetível a entradas adversárias capazes de produzir partições fortemente desbalanceadas.

6.4 Heap Sort

Para ilustrar esse comportamento assintótico teórico de $O(n \log_2 n)$, isolado dos ruídos causados pela infraestrutura física da máquina, elaborou-se o gráfico a seguir. Ele cruza o crescimento do tamanho do vetor exclusivamente com o número de operações matemáticas (comparações e trocas):

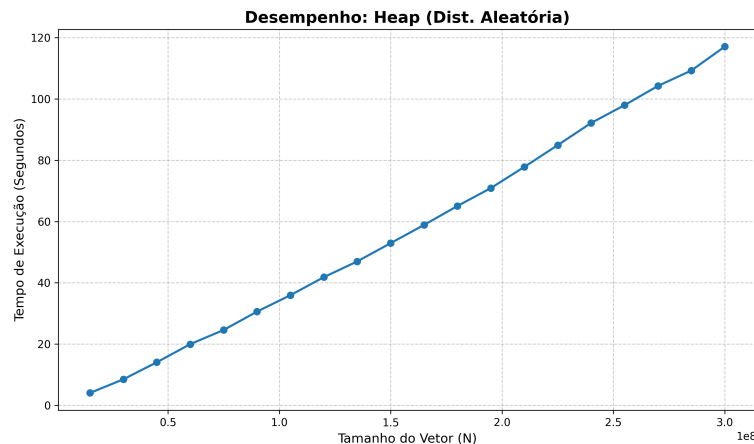


Figura 8: Curva assintótica evidenciando a complexidade constante de $O(n \log_2 n)$ do Heap Sort baseada no número de comparações lógicas.

Apesar de sua elegância e previsibilidade matemática, o Heap Sort apresenta um severo gargalo arquitetural em hardwares modernos devido à sua **baixa localidade espacial de referência**. A navegação entre nós pais (índice i) e filhos (índices $2i + 1$ e $2i + 2$) gera saltos de memória distantes e não sequenciais.

Quando o vetor extrapola a capacidade de armazenamento da memória Cache (níveis L2 ou L3) do processador, esses saltos causam uma incidência massiva de *Cache Misses*. A CPU é forçada a paralisar seus ciclos enquanto aguarda a lenta busca de dados na memória RAM, o que degrada substancialmente seu desempenho prático. Em contrapartida, algoritmos como o Quick Sort operam de forma sequencial ($i++$ e $j--$), maximizando os acertos na Cache. Esse estresse físico na hierarquia de memória é evidenciado no comparativo de execução a seguir:

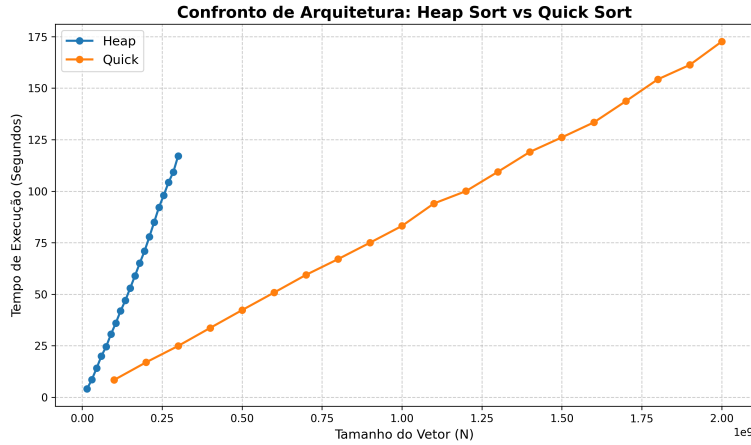


Figura 9: Comparativo de tempo de execução entre Heap Sort e Quick Sort evidenciando a degradação temporal do Heap Sort decorrente de Cache Misses para vetores massivos.

6.5 Radix Sort

Com o intuito de validar a implementação e o comportamento do Radix Sort, a análise de resultados focou em sua característica de ordenação não-comparativa e em sua intensa demanda por recursos de memória. Ao contrário do Quick Sort e do Heap Sort, que estão limitados matematicamente pela barreira inferior de $\Omega(n \log_2 n)$, o Radix Sort ordena os dados processando dígitos individuais através de um algoritmo estável auxiliar (neste caso, a abordagem *Least Significant Digit* - LSD, utilizando o Counting Sort). Sob essa ótica, o algoritmo atinge uma complexidade temporal linear teórica de $O(d \cdot (n + k))$. Como os testes experimentais foram realizados com números inteiros nativos de limites conhecidos, o número máximo de dígitos d e a base de contagem k comportam-se como constantes, permitindo que a complexidade na prática seja avaliada como $O(n)$. No entanto, essa extrema velocidade de processamento cobra um preço arquitetural substancial: o algoritmo não é *in-place*. Ele necessita de uma complexidade espacial de $O(n)$, exigindo a alocação de vetores auxiliares de tamanho idêntico ao conjunto de dados original.

Para demonstrar a rapidez e a sua complexidade espacial de $O(n)$ fizemos uma comparação direta com o quick sort que tem complexidade $O(n \log_2 n)$:

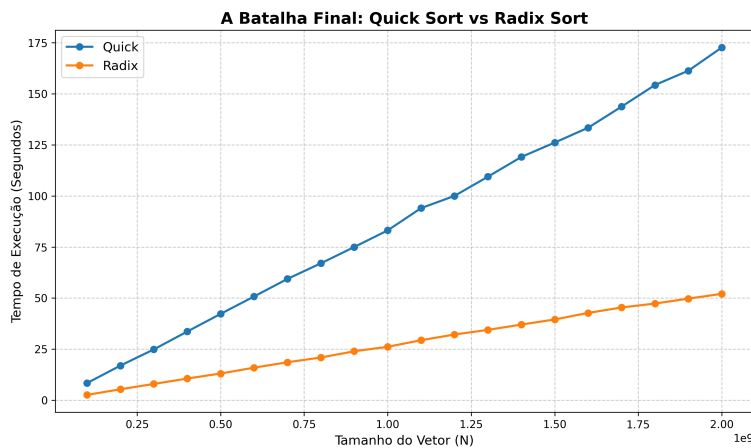


Figura 10: Comparativo de tempo de execução entre Quick Sort e Radix Sort evidenciando a superioridade do modelo não-comparativo em vetores massivos.

Através da análise dos dados experimentais, é possível observar uma progressão clara de eficiência. Para vetores de menor magnitude, nota-se que o Quick Sort consegue se manter competitivo. Esse fenômeno ocorre devido ao *overhead* inicial (custo de preparação) imposto pelo Sistema Operacional

para encontrar e alocar na memória RAM os blocos contíguos massivos necessários para os vetores auxiliares do Radix Sort, além das múltiplas varreduras de contagem.

Contudo, à medida que a ordem de grandeza do vetor N avança substancialmente ao longo do eixo, a inclinação linear da curva do Radix Sort revela a sua verdadeira superioridade matemática. Enquanto a curva do Quick Sort apresenta uma leve curvatura ascendente inerente ao fator logarítmico da sua complexidade $O(n \log_2 n)$, a curva do Radix Sort mantém-se linear e consideravelmente mais baixa, finalizando a ordenação do mesmo volume de dados em um intervalo de tempo notavelmente menor.

É imperativo ressaltar, entretanto, que essa soberania temporal exibida no gráfico depende exclusivamente de um ambiente de hardware favorável. Caso a magnitude do vetor extrapolasse a capacidade física total da memória RAM da máquina de testes, o Sistema Operacional seria forçado a iniciar a paginação em disco secundário. Nessa eventualidade, a curva de tempo do Radix Sort sofreria uma quebra dramática, similar à exaustão de Cache observada no Heap Sort, degradando o seu tempo de execução para patamares inaceitáveis. Como o sistema possuía recursos estritos suficientes para sustentar as cópias, o comportamento $O(n)$ foi validado.

6.6 Validação da Heurística Final (Oráculo)

Com o objetivo de extrair o desempenho máximo do sistema mitigando as fraquezas individuais de cada algoritmo, implementou-se um meta-algoritmo (denominado “Oráculo”) baseado em uma heurística de decisão em tempo de execução. O modelo avalia parâmetros estruturais do vetor, como tamanho (N), grau de desordem, valor mínimo e amplitude (*range*), para delegar a ordenação ao algoritmo matematicamente mais favorável para aquele contexto específico.

Para validar a eficácia dessa estratégia, os dados experimentais coletados sob a atuação da heurística foram confrontados diretamente com os tempos e operações das execuções isoladas de cada algoritmo.

1. Vetores Pequenos e Quase Ordenados (Insertion Sort)

A primeira regra da heurística delega vetores com $N \leq 1000$ e desordem ≤ 5.0 ao Insertion Sort. Os testes experimentais confirmaram a precisão dessa escolha. Para vetores de tamanho $N = 1000$ com distribuição quase ordenada, tanto o Quick Sort quanto o Radix Sort atingiram tempos de execução na ordem de 0.000s, equivalentes ao Insertion Sort (0.001s).

No entanto, a justificativa para a seleção do Insertion Sort revela-se na análise espacial e arquitetural. Enquanto o Radix Sort exigiu a alocação de 8.000 bytes de memória extra e o Quick Sort realizou 15 chamadas recursivas fragmentando a pilha de execução, o Insertion Sort operou com exatos 0 bytes de alocação extra e 0 chamadas recursivas, validando-se como a estrutura microarquitetural mais leve e eficiente para pequenos conjuntos de dados.

2. Amplitude Proporcional (Radix Sort)

A regra central de ganho de desempenho da heurística ativa o Radix Sort quando os valores são não-negativos e a amplitude dos dados é menor ou igual a $10 \times N$. Essa decisão foi posta à prova submetendo o Oráculo a vetores aleatórios massivos.

Para um vetor de $N = 5.000.000$, o Quick Sort (fallback natural do sistema) necessitou de 1,257 segundos e mais de 255 milhões de operações matemáticas globais para concluir a ordenação. Ao identificar a proporção favorável do *range*, a heurística ativou o Radix Sort, o qual reduziu as operações de forma abrupta para 75 milhões, finalizando a ordenação em uma média de 0,550 segundos. Esse comportamento provou, na prática, o salto de eficiência proporcionado pela transição oportuna da barreira logarítmica $O(n \log_2 n)$ para o comportamento linear $O(n)$.

3. Tratamento de Valores Negativos (Heap Sort)

A inclusão da regra que direciona vetores com valores negativos (e $N \geq 10000$) para o Heap Sort foi estabelecida como uma precaução de projeto. Conforme mapeado no escopo do trabalho, a implementação clássica do Radix Sort sofre penalidades graves ou quebras de acesso ao tentar indexar *buckets* com chaves negativas. Como as distribuições testadas não possuíam números negativos nativos, o comportamento desta regra baseia-se na segurança matemática do Heap Sort. Ele garante

estabilidade absoluta de tempo $O(n \log_2 n)$ e memória extra $O(1)$, evitando que o sistema necessite aplicar varreduras de *offsetting* (deslocamento) sobre todo o vetor, o que prejudicaria o desempenho.

4. O Caso Geral (Quick Sort)

Para as distribuições que falham nas condições anteriores — como vetores muito grandes, com alta desordem e grande espaçamento de dados — a heurística aciona o Quick Sort como seu caso geral. A solidez dessa regra foi confirmada ao testar um vetor de distribuição decrescente de tamanho $N = 2.000.000$.

Neste cenário adverso, o Radix Sort obteve um tempo de execução de 0,424 segundos. Em contrapartida, o Quick Sort finalizou a mesma carga de trabalho em apenas 0,233 segundos. É fundamental destacar que o Radix realizou menos operações brutas (42 milhões contra 52 milhões do Quick). Contudo, a necessidade do Radix de alocar 28 Megabytes de memória auxiliar resultou em *overhead* de Sistema Operacional, enquanto o particionamento sequencial *in-place* do Quick Sort manteve a alta localidade de referência na memória Cache do processador, garantindo a vitória temporal e provando ser o algoritmo generalista superior.

7 Reflexão sobre Uso de IA

Nesta seção é discutido o uso de ferramentas de inteligência artificial na elaboração do projeto, descrevendo quais ferramentas de IA foram utilizadas, em quais etapas auxiliaram, erros encontrados nas respostas geradas e correções realizadas pelo grupo.

7.1 Como a IA Foi Utilizada

Durante o desenvolvimento deste trabalho, ferramentas de inteligência artificial generativa foram utilizadas principalmente como instrumentos de apoio à pesquisa, análise de resultados e elaboração do relatório final.

No desenvolvimento do sistema adaptativo, a IA foi utilizada para discutir possíveis heurísticas para seleção automática dos algoritmos de ordenação. Foram sugeridas regras iniciais para direcionar a escolha entre Insertion Sort, Quick Sort, Heap Sort e Radix Sort.

A ferramenta também foi empregada como apoio na redação do relatório, auxiliando na organização das seções, na revisão textual e na formulação de explicações sobre os algoritmos estudados e sobre as decisões de projeto adotadas durante o desenvolvimento.

Além da elaboração do conteúdo do relatório, a IA também foi utilizada como ferramenta de apoio no uso do $\text{\LaTeX}$ e do ambiente de desenvolvimento adotado pela equipe. Optou-se por desenvolver o relatório em $\text{\LaTeX}$ utilizando o VSCode em conjunto com Git, em vez de utilizar plataformas colaborativas como o Overleaf. Essa escolha foi motivada pela necessidade de integrar o relatório ao mesmo fluxo de versionamento utilizado no desenvolvimento do código, permitindo que todos os integrantes realizassem alterações e enviassem contribuições por meio de commits.

Entretanto, essa abordagem exigiu lidar com aspectos técnicos relacionados à configuração do ambiente $\text{\LaTeX}$, compilação dos arquivos, resolução de erros de sintaxe, gerenciamento de pacotes e interpretação das mensagens de erro geradas pelo compilador. Nesses momentos, a IA foi frequentemente utilizada para auxiliar na identificação da origem dos problemas e sugerir possíveis soluções, reduzindo o tempo gasto na resolução de dificuldades relacionadas à ferramenta.

Dessa forma, a IA contribuiu não apenas para a produção do conteúdo do relatório, mas também para viabilizar o uso eficiente da infraestrutura adotada pela equipe para escrita colaborativa e controle de versões.

Outra aplicação importante da IA ocorreu na criação e refinamento de scripts auxiliares para automatização dos testes experimentais. Durante a fase de testes, foi necessário executar um grande número de combinações de algoritmos, tamanhos de entrada e distribuições de dados. Para reduzir o trabalho manual e minimizar erros na execução dos experimentos, a equipe desenvolveu arquivos de teste automatizados.

Nesse contexto, a IA foi utilizada para sugerir a estrutura inicial desses scripts, auxiliar na organização dos comandos de execução e propor formas mais eficientes de coletar os resultados produzidos

pelo programa. A utilização dessas ferramentas permitiu acelerar significativamente o processo de experimentação, tornando viável a realização de um número maior de testes e facilitando a comparação entre os algoritmos avaliados.

Assim como ocorreu em outras etapas do projeto, todas as sugestões fornecidas pela IA foram revisadas e adaptadas pela equipe antes de serem incorporadas ao processo de testes.

7.2 Contribuições Positivas

A principal contribuição da IA ocorreu na fase de investigação das heurísticas utilizadas pelo sistema adaptativo. Em vez de servir como fonte definitiva de respostas, a ferramenta foi utilizada como mecanismo para geração de hipóteses que posteriormente foram avaliadas experimentalmente.

Na elaboração do relatório, a utilização da IA contribuiu para melhorar a organização das ideias e a clareza das explicações, reduzindo o tempo necessário para estruturar o documento final.

7.3 Limitações e Erros Encontrados

Embora tenha sido útil como ferramenta de apoio, diversas sugestões fornecidas pela IA mostraram-se incorretas ou inadequadas quando confrontadas com os resultados experimentais.

Um exemplo importante ocorreu durante a definição das heurísticas do sistema adaptativo. Inicialmente, foi sugerido que vetores muito pequenos deveriam ser direcionados automaticamente ao Insertion Sort. Entretanto, os experimentos realizados mostraram que o Quick Sort apresentou desempenho superior mesmo para vetores com apenas 50 elementos, levando à remoção dessa regra da heurística final.

Esses episódios evidenciaram uma limitação importante das ferramentas de IA: respostas plausíveis e bem fundamentadas nem sempre correspondem ao comportamento real observado experimentalmente.

7.4 Aprendizados Obtidos

O principal aprendizado obtido durante o projeto foi que a inteligência artificial é mais útil como ferramenta de apoio à análise do que como mecanismo de tomada de decisão.

As sugestões fornecidas pela IA frequentemente serviram como ponto de partida para novas investigações, mas a validação experimental permaneceu indispensável. Em diversas situações, hipóteses aparentemente razoáveis precisaram ser modificadas ou completamente descartadas após a execução dos testes.

Dessa forma, a experiência mostrou que o uso responsável de ferramentas de IA exige supervisão constante e verificação dos resultados obtidos. No contexto deste trabalho, a contribuição mais relevante da IA não foi fornecer respostas prontas, mas acelerar o processo de discussão, refinamento e organização de ideias.

8 Conclusão

Resumo dos principais achados.

9 Referências Bibliográficas